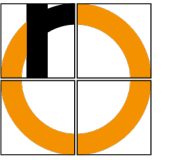


Theoretical Computer Science

Word Problem & Parsing

Technische Hochschule Rosenheim
Sommer 2026
Prof. Dr. Jochen Schmidt

- Solving the word problem
- CYK-Parser
- Application: Compilers



Analysis of Words & Parsing

- Word problem: Decide whether a word x is well-formed (i.e., part of given language)
- Parsing problem (*Zerteilungsproblem*)
 - complete analysis of the word x
 - derivation of x is traced back to the start symbol S or vice versa
 - all steps from $S \Rightarrow^* x$ must be determined
- Questions:
 - How do we solve these problems for the various Chomsky language classes?
 - How difficult/time-consuming is the decision?

Word Problem for Regular Languages (Type 3)

- Create a **deterministic finite automaton** for the language
- If the processing stops in an end state, the analyzed word is part of the language
- Time complexity: **linear** with respect to the word length

- Create a pushdown automaton? We may require a **nondeterministic** PDA for type 2 ...
- Better: Convert grammar to **Chomsky normal form** (*Chomsky-Normalform*) and use the **CYK-Algorithm** (Cocke, Younger, Kasami)
- Time complexity: $O(n^3)$ with respect to the word length
 - too slow for practical purposes (e.g., syntax analysis in a compiler)
 - in programming languages: restriction to deterministic context-free languages and LR(k) grammars
 - these have linear time complexity
 - and we can use a deterministic PDA for solving the word problem

- For any context-free grammar G with $\varepsilon \notin L(G)$ we can construct a grammar G' with $L(G) = L(G')$ that is in Chomsky normal form
- A grammar is **Chomsky normal form (CNF)** if all production rules have the form (A, B, C : syntactic variables, a : terminal symbol)
 - $A \rightarrow BC$
 - $A \rightarrow a$
- If $\varepsilon \in L(G)$: Add the rule $S \rightarrow \varepsilon$, S must not appear on the right side of any production
 - this is already required by our definition of type 2 grammars

- For each terminal symbol a : Create a new variable V_a
 - add productions $V_a \rightarrow a$ to the grammar
 - replace terminal symbols a on all right-hand sides by V_a
- Replace rules with more than 2 variables on the right
 - the rule $A \rightarrow B_1 B_2 \dots B_k$, $k \geq 3$ becomes:
 - $A \rightarrow B_1 V_2$
 - $V_2 \rightarrow B_2 V_3$
 - ...
 - $V_{k-1} \rightarrow B_{k-1} B_k$
- Replace productions of the form $A \rightarrow B$
 - remove all rules $A \rightarrow B$
 - for each rule $B \rightarrow b$ add a rule $A \rightarrow b$
 - for each rule $B \rightarrow XY$ add a rule $A \rightarrow XY$

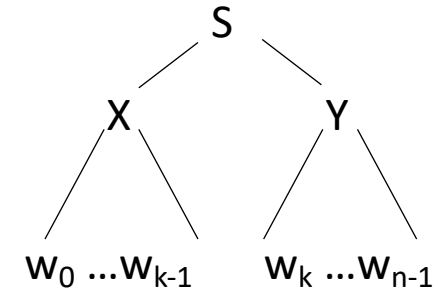
Example: $L = \{a^m b^n c^n \mid m, n \geq 1\}$
 $S \rightarrow AB, A \rightarrow Aa \mid a, B \rightarrow bBc \mid bc$

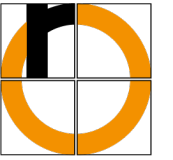
$V_a \rightarrow a, V_b \rightarrow b, V_c \rightarrow c$
 $S \rightarrow AB, A \rightarrow AV_a \mid V_a, B \rightarrow V_b BV_c \mid V_b V_c$

$V_a \rightarrow a, V_b \rightarrow b, V_c \rightarrow c$
 $S \rightarrow AB, A \rightarrow AV_a \mid V_a, B \rightarrow V_b V_1 \mid V_b V_c,$
 $V_1 \rightarrow BV_c$

$V_a \rightarrow a, V_b \rightarrow b, V_c \rightarrow c$
 $S \rightarrow AB, A \rightarrow AV_a \mid a, B \rightarrow V_b V_1 \mid V_b V_c,$
 $V_1 \rightarrow BV_c$

- Given: Word of length n , $w = w_0w_1\dots w_{n-1} \in \Sigma^*$
- Case $n = 1$, i.e., $w = w_0$
 - As grammar is in CNF: Rule $S \rightarrow w_0$ must exist
- Case $n > 1$
 - As grammar is in CNF: the word must consist of 2 subwords
 - These can be derived via a production $S \rightarrow XY$
 - The parsing problem has now been reduced to 2 subwords of length k and $n - k$
 - k is unknown, i.e., we have to look at all possible split positions
 - We now apply the same procedure to both subwords, until we get subwords of length 1
- A table of size $n \times n$ is required, but only half of the entries are occupied
- Instead of top-down, CYK operates bottom-up: It starts at subwords of length 1

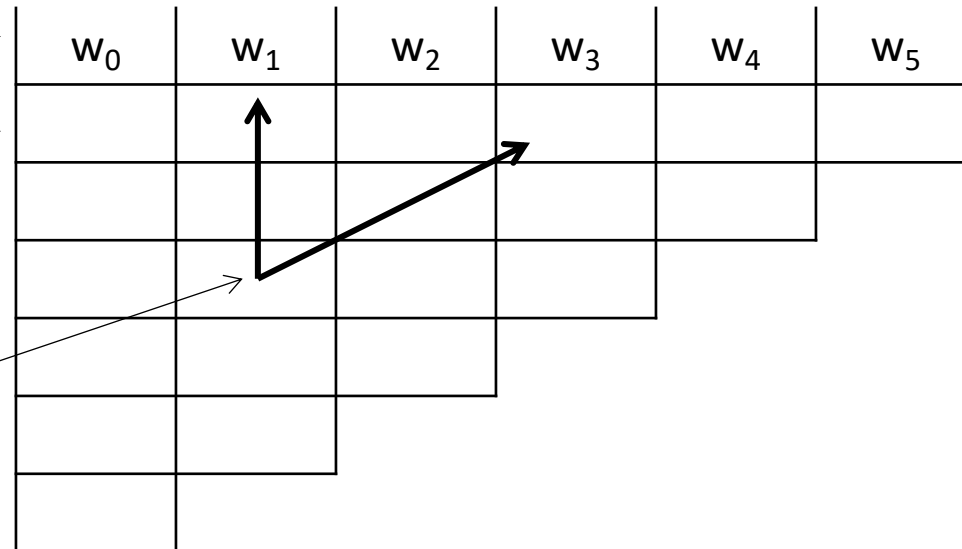




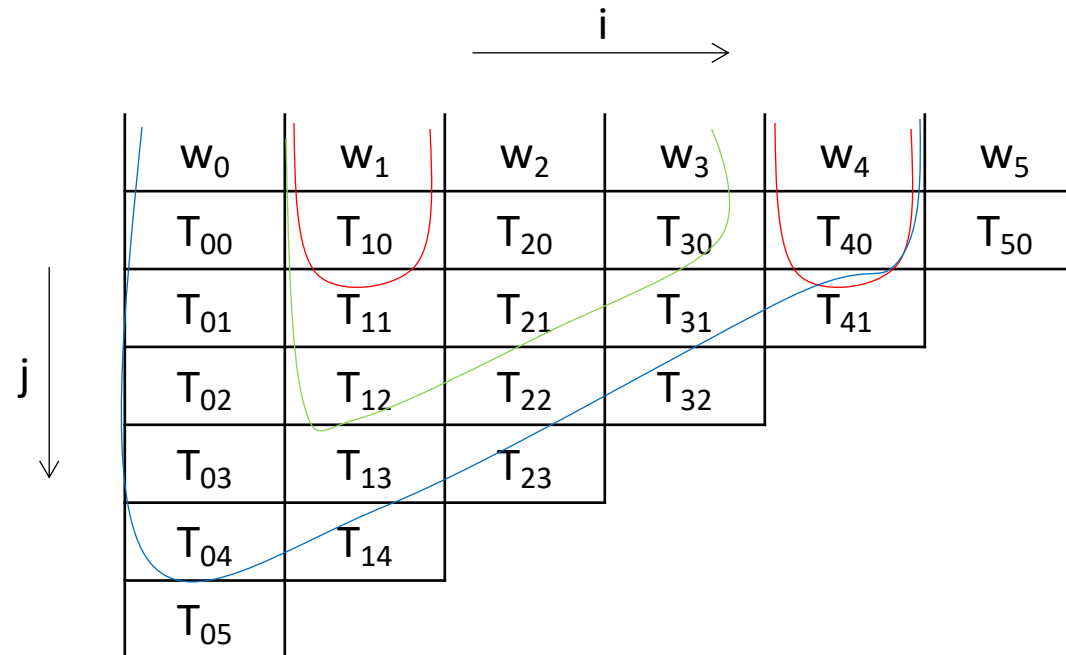
Start with word in top row (not stored in table)

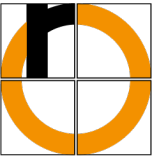
In each field of the 1st row,
enter all the variables from
which w_i can be generated

For each entry: Check vertically and
diagonally to see if there is a rule at
the correct distance that generates the
subword; if yes: enter the variable(s)



What set of variables in the entry T_{ij} generates which subword:

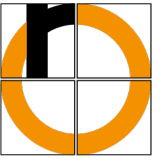




Which rules are considered? Search vertically and diagonally, but at the right distance!

$i \rightarrow$

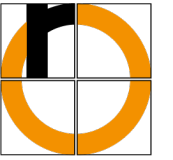
	w_0	w_1	w_2	w_3	w_4	w_5
$j \downarrow$	T_{00}	T_{10}	T_{20}	T_{30}	T_{40}	T_{50}
	T_{01}	T_{11}	T_{21}	T_{31}	T_{41}	
	T_{02}	T_{12}	T_{22}	T_{32}		
	T_{03}	T_{13}	T_{23}			
	T_{04}	T_{14}				
	T_{05}					



Which rules are considered? Search vertically and diagonally, but at the right distance!

$i \rightarrow$

	w_0	w_1	w_2	w_3	w_4	w_5
	T_{00}	T_{10}	T_{20}	T_{30}	T_{40}	T_{50}
	T_{01}	T_{11}	T_{21}	T_{31}	T_{41}	
	T_{02}	T_{12}	T_{22}	T_{32}		
$j \downarrow$	T_{03}	T_{13}	T_{23}			
	T_{04}	T_{14}				
	T_{05}					



Which rules are considered? Search vertically and diagonally, but at the right distance!

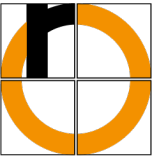
$i \rightarrow$

	w_0	w_1	w_2	w_3	w_4	w_5
	T_{00}	T_{10}	T_{20}	T_{30}	T_{40}	T_{50}
	T_{01}	T_{11}	T_{21}	T_{31}	T_{41}	
$j \downarrow$	T_{02}	T_{12}	T_{22}	T_{32}		
	T_{03}	T_{13}	T_{23}			
	T_{04}	T_{14}				
	T_{05}					

Which rules are considered? Search vertically and diagonally, but at the right distance!

$i \rightarrow$

	w_0	w_1	w_2	w_3	w_4	w_5
	T_{00}	T_{10}	T_{20}	T_{30}	T_{40}	T_{50}
	T_{01}	T_{11}	T_{21}	T_{31}	T_{41}	
	T_{02}	T_{12}	T_{22}	T_{32}		
	T_{03}	T_{13}	T_{23}			
	T_{04}	T_{14}				
	T_{05}					
$j \downarrow$						



Which rules are considered? Search vertically and diagonally, but at the right distance!

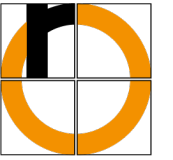
$i \rightarrow$

	w_0	w_1	w_2	w_3	w_4	w_5
$j \downarrow$	T_{00}	T_{10}	T_{20}	T_{30}	T_{40}	T_{50}
	T_{01}	T_{11}	T_{21}	T_{31}	T_{41}	
	T_{02}	T_{12}	T_{22}	T_{32}		
	T_{03}	T_{13}	T_{23}			
	T_{04}	T_{14}				
	T_{05}					

- Given: Grammar G in CNF $S \rightarrow AB, A \rightarrow CD \mid CF, B \rightarrow c \mid EB$
 $C \rightarrow a, D \rightarrow b, E \rightarrow c, F \rightarrow AD$ start symbol S
- Is $w = aaabbbcc \in L(G)$?

a	a	a	b	b	b	c	c
C	C	C	D	D	D	B, E	B, E

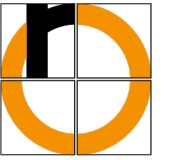
CYK-Algorithm – Example



$S \rightarrow AB, A \rightarrow CD \mid CF, B \rightarrow c \mid EB$
 $C \rightarrow a, D \rightarrow b, E \rightarrow c, F \rightarrow AD$

a	a	a	b	b	b	c	c
C	C	C	D	D	D	B, E	B, E
		A				B	

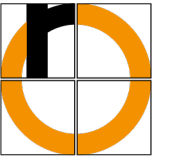
CYK-Algorithm – Example



$S \rightarrow AB, A \rightarrow CD \mid CF, B \rightarrow c \mid EB$
 $C \rightarrow a, D \rightarrow b, E \rightarrow c, F \rightarrow AD$

a	a	a	b	b	b	c	c
C	C	C	D	D	D	B, E	B, E
		A				B	
		F					

CYK-Algorithm – Example



$S \rightarrow AB, A \rightarrow CD \mid CF, B \rightarrow c \mid EB$
 $C \rightarrow a, D \rightarrow b, E \rightarrow c, F \rightarrow AD$

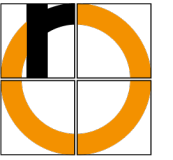
a	a	a	b	b	b	c	c
C	C	C	D	D	D	B, E	B, E
		A				B	
		F					
	A						
	F						
A							
S							
S							

- Decidable, since the grammar must be monotonic:
For a word of length n all intermediate results cannot be longer than n symbols
- The number of words with length n over a finite alphabet is finite

Therefore, there must exist an algorithm that solves the word problem:

- Try out all possible derivatives
- Time complexity: $O(a^n)$ with respect to the word length
 - not suitable for practical purposes
 - (a = number of symbols of the alphabet)

- Grammars can have dead-end derivations
 - for **type 1** languages it is guaranteed that a dead-end has finite length
 - for **type 0** languages a dead-end can also be **infinitely** long (the grammar is not necessarily monotonic)
- The word problem for type 0 languages is unsolvable!
 - **There exists no algorithm** that can decide for all type 0 languages whether a word w is generated by a given type 0 grammar or not
 - We say the problem is **undecidable** (*unentscheidbar*)



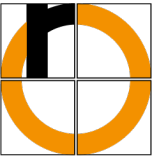
Application: Compilers

- **Compiler** (*Übersetzer*):
A program that translates the instructions of a program written in one programming language P1 (**source language**) into instructions of another programming language P2 (**target language**)
- Source program $a \in P1$ must be semantically equivalent to target program $b \in P2$ (they must do the same thing)
- Formal languages are used for this purpose

- Compilers in the narrower sense
 - Source language P1 is a higher programming language compared to the target language P2
- Assembler
 - Compiler for transferring assembly language (assembler lang.) source programs to machine language
- Cross-Compiler
 - Compiler generates target code that runs on a different platform than the compiler itself
 - other operating system and/or CPU
- Pre-processor/Pre-compiler
 - Translation of language extensions before actual compilation (typically used in C)
- Compiler-Compiler
 - Program for generating a compiler from a formalized language description
 - e.g.: YACC (Yet Another Compiler Compiler)

- Interpreter
 - Instructions of the source code are translated and executed immediately
 - Advantage:
 - Tests during development very quick, without a separate compilation step
 - important instrument if program is to run without modification on computers with different operating systems and different hardware
 - Disadvantage: The translation time is always added to the execution time while the program runs
 - costs a lot of time, especially for loops
 - Examples: BASIC, LISP, PROLOG, Python; with restrictions: Java

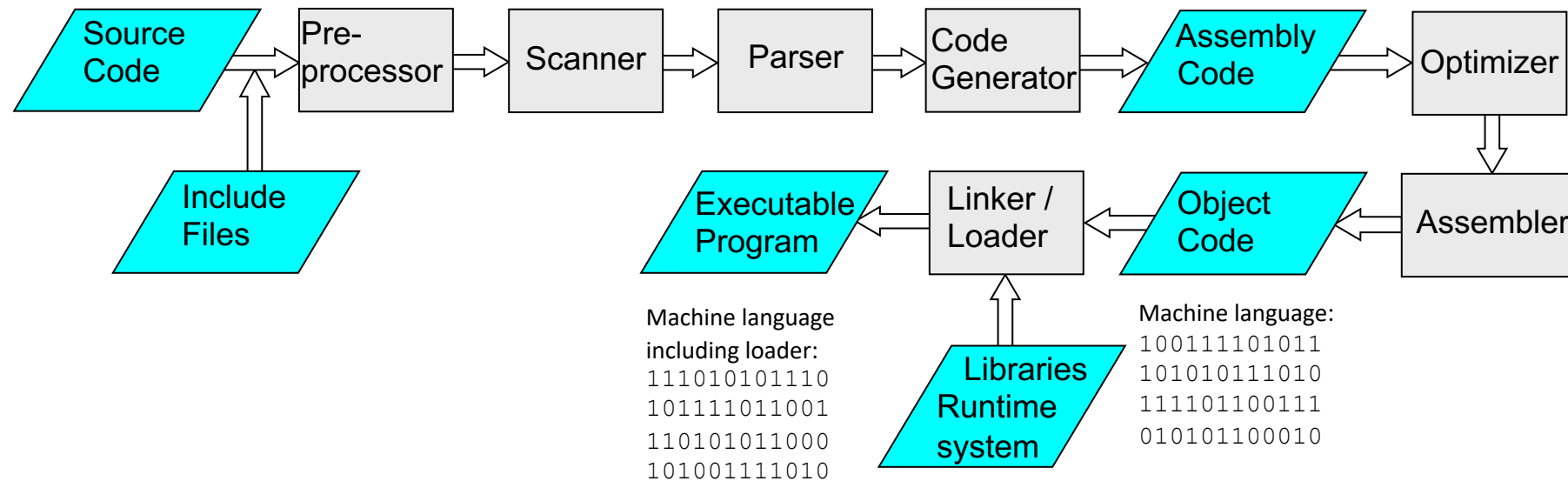
Compiler: Steps, shown for C Language



```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello world\n");  
}
```

```
...  
movebp, esp  
addesp, 204; 000000ccH  
cmpebp, esp  
...
```



Example: Source Code → Pre-processor

```
#include <stdio.h> // Standard Input/ Output z.B. scanf, printf
#define TEXT "Dies ist ein Text\n"

int main(void)
{
    const float zahl = 5.2f; // definiert eine Konstante

    printf ("Zahl: %10.2f\n", zahl); /* Ausgabe */
    printf(TEXT);

    return(0);
}
```

Original source code (complete)

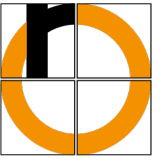
```
...
#line 283 "c:\\program files (x86)\\microsoft visual studio 10.0\\vc\\include\\stdio.h"
__declspec(dllimport) int __cdecl _pclose( FILE * _File);
__declspec(dllimport) FILE * __cdecl _popen( const char * _Command, const char * _Mode);
__declspec(dllimport) int __cdecl printf( const char * _Format, ...);
...

int main(void)
{
    const float zahl = 5.2f;
    printf ("Zahl: %10.2f\n", zahl);
    printf("Dies ist ein Text\n");

    return(0);
}
```

after pre-processor
(small excerpt)

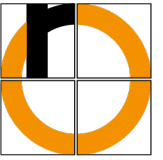
Example: Assembly Code (Excerpt)



```
_TEXTSEGMENT
_zahl$ = -8; size = 4
_mainPROC; COMDAT
; Line 5
pushebp
movebp, esp
subesp, 204; 000000cch
pushebx
pushesi
pushedi
leaedi, DWORD PTR [ebp-204]
movecx, 51; 00000033H
moveax, -858993460; ccccccccH
rep stosd
; Line 6
fldDWORD PTR __real@40a66666
fstpDWORD PTR _zahl$[ebp]
; Line 8
fldDWORD PTR _zahl$[ebp]
movesi, esp
subesp, 8
fstpQWORD PTR [esp]
pushOFFSET
```

```
??_C@_00@KIDDCBJA@Zahl?3?5?$CF10?42f?6?$AA@
callDWORD PTR __imp__printf
addesp, 12; 0000000cH
cmpesi, esp
call__RTC_CheckEsp
; Line 9
movesi, esp
pushOFFSET
??_C@_0BD@0PJJLBBI@Dies?5ist?5ein?5Text?6?$AA@
callDWORD PTR __imp__printf
addesp, 4
cmpesi, esp
call__RTC_CheckEsp
; Line 11
xoreax, eax
; Line 12
popedi
popesi
popebx
addesp, 204; 000000cch
cmpebp, esp
call__RTC_CheckEsp
movesp, ebp
popebp
ret0
_mainENDP
_TEXTENDS
```

Example: Object File (Excerpt)



Line number
(not part of the file)



```
00000f80 02 00 00 00 00 00 87 00 00 00 00 00 00 00 00 00 .....rtc$TMZ....
00000f90 20 00 02 00 2E 72 74 63 24 54 4D 5A 00 00 00 00 .....L.....
00000fa0 08 00 00 00 03 02 04 00 00 00 01 00 00 00 00 00 .....
00000fb0 00 00 03 00 05 00 00 00 CC 11 2E 4C 00 00 00 00 .....
00000fc0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 96 00 .....
00000fd0 00 00 00 00 00 00 08 00 00 00 03 00 00 00 00 00 .....
00000fe0 AD 00 00 00 00 00 00 00 00 00 20 00 02 00 2E 72 .....r
00000ff0 74 63 24 49 4D 5A 00 00 00 00 09 00 00 00 03 02 tc$IMZ.....
00001000 04 00 00 00 01 00 00 00 00 00 00 00 03 00 05 00 .....
00001010 00 00 9E 7A 90 5D 00 00 00 00 00 00 00 00 00 00 ...z.].....
00001020 00 00 00 00 00 00 00 00 BC 00 00 00 00 00 00 00 .....
00001030 09 00 00 00 03 00 00 00 00 00 D3 00 00 00 00 00 .....
00001040 00 00 00 00 20 00 02 00 2E 64 65 62 75 67 24 54 .....debug$T
00001050 00 00 00 00 0A 00 00 00 03 02 74 00 00 00 00 00 .....t.....
00001060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 E2 00 .....
00001080 00 00 3F 3F 5F 43 40 5F 30 42 44 40 4F 50 4A 4A ...??_C@_OBD@OPJJ
00001090 4C 42 42 49 40 44 69 65 73 3F 35 69 73 74 3F 35 LBBI@Dies?5ist?5
000010a0 65 69 6E 3F 35 54 65 78 74 3F 36 3F 24 41 41 40 ein?5Text?6?5AA@
000010b0 00 5F 5F 69 6D 70 5F 5F 70 72 69 6E 74 66 00 3F ...imp_printf.?
000010c0 3F 5F 43 40 5F 30 4F 40 4B 49 44 44 43 42 4A 41 ?_C@_00@KIDDCBJA
000010d0 40 5A 61 68 6C 3F 33 3F 35 3F 24 43 46 31 30 3F @Zahl?3?5?5CF10?
000010e0 34 32 66 3F 36 3F 24 41 41 40 00 5F 5F 72 65 61 42f?6?5AA@...rea
000010f0 6C 40 34 30 61 36 36 36 36 36 00 5F 5F 66 6C 74 l@40a66666...flt
00001100 75 73 65 64 00 5F 5F 52 54 43 5F 43 68 65 63 6B used...RTC_Check
00001110 45 73 70 00 5F 5F 52 54 43 5F 53 68 75 74 64 6F Esp...RTC_Shutdo
00001120 77 6E 2E 72 74 63 24 54 4D 5A 00 5F 5F 52 54 43 wn.rtc$TMZ...RTC
00001130 5F 53 68 75 74 64 6F 77 6E 00 5F 5F 52 54 43 5F _Shutdown...RTC
00001140 49 6E 69 74 42 61 73 65 2E 72 74 63 24 49 4D 5A InitBase.rtc$IMZ
00001150 00 5F 5F 52 54 43 5F 49 6E 69 74 42 61 73 65 00 ...RTC_InitBase.
```

Machine code
(hexadecimal)

Interpretation
as ASCII

- **Lexical analysis** (*lexikalische Analyse*)
 - Conversion of the source program $a \in P1$ with **scanner** into intermediate code (**token**)
 - Objects of the language (e.g., operators, keywords, identifiers) are recognized as such and converted into tokens
 - Simple rule violations can already be reported here
 - e.g., use of an illegal character in an identifier
 - Description by regular grammar/regular expressions
 - Implemented as **deterministic finite automaton**

- **Syntactic analysis** (*syntaktische Analyse*)
 - **Parser** generates the syntax tree of the program $a \in P1$ from tokens according to the syntax of $P1$
 - Uses deterministic context-free grammars
 - Top-Down: LL(k) grammar, usually LL(1)
 - Bottom-Up: LR(k) grammar, usually LR(1)
 - Implemented as deterministic pushdown automaton
- **Semantic analysis** (*semantische Analyse*)
 - Analysis of the syntax tree of $a \in P1$: Check the semantics of the program, e.g.,
 - Have all variables used been declared?
 - Are they used according to their type?
 - Are there violations of range limits?
 - Simultaneously: Code generator transfers a to the target language $P2$
 - Result: Target program $b \in P2$
 - Uses (context-free) **attribute grammars** (*Attributgrammatiken*)

- **Code Optimization**

- Goal: Increase efficiency of the target program $b \in P2$
- Optimize **runtime** and/or **memory** requirements; usually this is a trade-off
- Code optimization is time-consuming
- Program code $b \in P2$ is modified
 - Semantics, of course, should remain unchanged
 - Problem: Complete preservation of the semantics of b cannot be guaranteed in every case
- Therefore: optional step, with various degrees of more or less aggressive optimization

- lex / flex: Lexical analysis
 - lex: 1975
- yacc / bison: Parser – syntactic/semantic analysis
 - yacc: 1979
- Generate C code files
- Links:
 - lex & yacc: <http://dinosaur.compilertools.net/>
 - flex: <http://flex.sourceforge.net/>
 - bison: <http://www.gnu.org/software/bison/>
 - Overview Scanner/Parser generators:
https://en.wikipedia.org/wiki/Comparison_of_parser_generators

“The asteroid to kill this dinosaur is still in orbit.”
(Lex Manual Page)

- Word problem
 - regular language: finite automaton
 - context-free language: CYK algorithm
 - context-sensitive language: brute-force – try out all possibilities
 - Type 0 language: no algorithm exists (undecidable)
- Main steps during compilation of a program
 - lexical analysis
 - scanner
 - conversion to tokens
 - regular grammar/DFA
 - syntactic analysis
 - parser
 - generation of a syntax tree
 - deterministic context-free grammar/DPDA
 - semantic analysis
 - Analysis of syntax tree
 - context-free attribute grammars
 - Code generation and optimization
 - Linking

- H. Ernst, J. Schmidt und G. Beneken: *Grundkurs Informatik*. Springer Vieweg, 8. Aufl., 2023.
- Schöning, U.: *Theoretische Informatik – kurz gefasst*. Spektrum Akad. Verlag (2008)
- Sander P., Stucky W., Herschel, R.: *Automaten, Sprachen, Berechenbarkeit*, B.G. Teubner, 1992